



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

CARRERA:

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

NRC: 30693

ASIGNATURA:

METODOLOGÍA DE DESARROLLO DE SOFTWARE

INTEGRANTES:

CAYO JANINE

CHAFLA SANTIAGO

GUAYCHA ANTHONY

MADRID EDMIR

DOCENTE:

ING. JENNY RUIZ, MGT.

GUÍA PARA DESARROLLAR EN PLANTUML DIAGRAMAS DE CASOS DE USO DE ANÁLISIS

Ejemplo base: CRUD de Estudiante

Asignatura: Metodología de desarrollo de Software

Tema: Modelado de casos de uso de nivel 0 y apoyo al análisis orientado a objetos con PlantUML

Caso base: Gestión de estudiantes con los datos ID, Nombre y Edad, según la interfaz CRUD de Estudiantes presentada en la imagen de referencia.

Propósito: orientar al estudiante para pasar desde requisitos funcionales de nivel 0 hacia casos de uso claros, trazables y verificables.

1. Propósito de la guía

Esta guía orienta el desarrollo de diagramas de casos de uso de análisis en PlantUML, tomando como referencia una secuencia académica progresiva: partir de una ERS/SRS, seleccionar requisitos funcionales de nivel 0, identificar actores, modelar casos de uso principales y mantener trazabilidad entre requisito, actor, caso de uso y criterio de aceptación.

El ejemplo se centra en un CRUD de Estudiante. Para efectos de análisis, CRUD se interpreta como el conjunto de funcionalidades principales que permiten agregar, mostrar, actualizar y eliminar información de estudiantes.

2. REQUISITOS FUNCIONALES de nivel 0 para el CRUD

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir agregar un nuevo estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente usando ID, Nombre y Edad.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante seleccionado cuando corresponda según la regla académica definida.	Administrador académico	Eliminar estudiante

RF-04	El sistema debe permitir mostrar todos los estudiantes registrados en una tabla con las columnas ID, Nombre y Edad.	Administrador académico	Mostrar todo
RF-05	El sistema debe validar los datos obligatorios antes de guardar o actualizar cambios.	Sistema	Validar datos del estudiante

3. DIAGRAMA DE CASOS DE USO

Código PlantUML:

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador académico" as Admin

rectangle "Sistema CRUD de Estudiantes" {
    usecase "Agregar estudiante" as UC1
    usecase "Actualizar estudiante" as UC2
    usecase "Eliminar estudiante" as UC3
    usecase "Mostrar todo" as UC4
    usecase "Validar datos del estudiante" as UC5
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

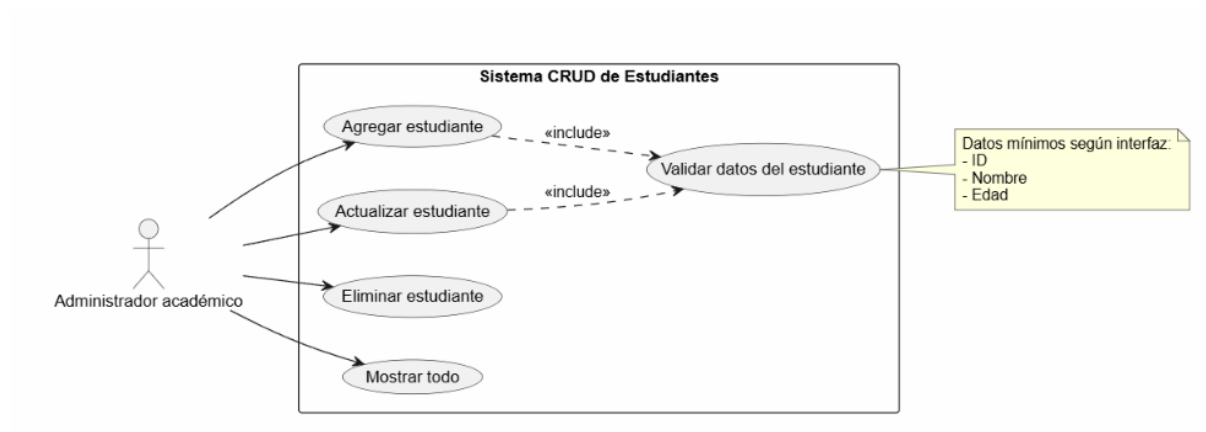
UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>

note right of UC5
Datos mínimos según interfaz:
- ID
- Nombre
- Edad

```

end note
@endum1

IMAGEN DEL DIAGRAMA



Interpretación:

El diagrama representa el funcionamiento básico del Sistema CRUD de Estudiantes, donde el actor principal es el Administrador académico, quien interactúa con las principales funciones del sistema.

El sistema permite agregar, actualizar, eliminar y mostrar estudiantes registrados. Para las opciones de agregar y actualizar, el sistema incluye obligatoriamente el proceso de validar datos del estudiante, asegurando que la información ingresada sea correcta antes de guardar cambios.

Los datos mínimos requeridos por la interfaz son: ID, Nombre y Edad. Además, el caso de uso Mostrar todo permite visualizar todos los registros almacenados en una tabla.

Cada caso de uso se relaciona directamente con los requisitos funcionales definidos para el CRUD, garantizando el correcto manejo de la información estudiantil.

4. Especificación breve de casos de uso

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Agregar estudiante	Administrador académico	El estudiante no debe existir previamente con el mismo ID.	Ingresa ID, Nombre y Edad -> validar datos -> guardar registro.	Estudiante agregado correctamente.

Actualizar estudiante	Administrador académico	El estudiante debe existir.	Ingresar o seleccionar ID -> modificar Nombre y/o Edad -> validar datos -> guardar cambios.	Datos del estudiante actualizados.
Eliminar estudiante	Administrador académico	El estudiante debe existir y cumplir regla para eliminación.	Seleccionar o ingresar ID -> confirmar eliminación -> eliminar registro.	Estudiante eliminado o marcado como eliminado.
Mostrar todo	Administrador académico	Debe existir al menos un registro o una tabla vacía disponible.	Seleccionar Mostrar Todo -> mostrar registros -> presentar tabla.	Listado visible con ID, Nombre y Edad.

5. Puente hacia clases de análisis: frontera, control y entidad

Aunque esta guía se enfoca en casos de uso, los documentos base indican que el análisis orientado a objetos puede continuar identificando clases de frontera, control y entidad. Esto ayuda a preparar el paso hacia U1T2 y U1T3.

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante	Sintaxis PlantUML sugerida
Frontera / Boundary	Representa la interacción con el actor o una pantalla del sistema.	PantallaCRUDEstudiantes	boundary "Pantalla CRUD Estudiantes" as UI
Control / Control	Coordina la lógica del caso de uso y la comunicación entre frontera y entidad.	ControlEstudiante	control "Control Estudiante" as Ctrl
Entidad / Entity	Representa información del dominio que el sistema conserva.	Estudiante	entity "Estudiante" as Est

Código complementario para visualizar el puente entre casos de uso y clases de análisis:

Código PlantUML:

```
@startuml
left to right direction
skinparam shadowing false
actor "Administrador académico" as Admin

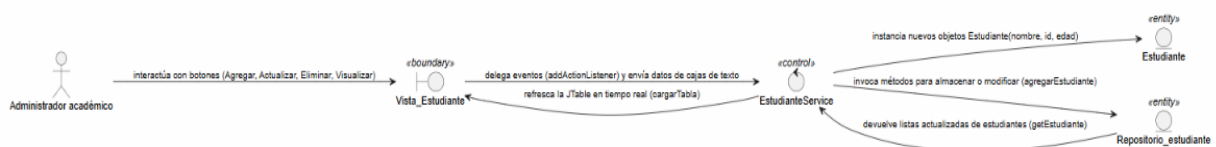
' Componentes reales de tu software mapeados a los estereotipos de análisis
boundary "Vista_Estudiante" as UI <<boundary>>
control "EstudianteService" as Ctrl <<control>>
entity "Estudiante" as Est <<entity>>
entity "Repositorio_estudiante" as Repo <<entity>>

' Flujo de interacciones dinámicas de tu código
Admin --> UI : interactúa con botones (Agregar, Actualizar, Eliminar, Visualizar)

UI --> Ctrl : delega eventos (addActionListener) y envía datos de cajas de texto
Ctrl --> Est : instancia nuevos objetos Estudiante(nombre, id, edad)
Ctrl --> Repo : invoca métodos para almacenar o modificar (agregarEstudiante)
Repo --> Ctrl : devuelve listas actualizadas de estudiantes (getEstudiante)
Ctrl --> UI : refresca la JTable en tiempo real (cargarTabla)

@enduml
```

IMAGEN DEL DIAGRAMA



Interpretación:

El diagrama de robustez muestra la interacción entre el Administrador académico y los componentes del sistema CRUD de estudiantes siguiendo el patrón Boundary-Control-Entity.

El actor interactúa primero con la interfaz Vista_Estudiante, donde utiliza los botones de agregar, actualizar, eliminar y visualizar estudiantes. Luego, la vista envía los eventos al componente de control EstudianteServicio, encargado de procesar la lógica del sistema.

El controlador se comunica con la entidad Estudiante, donde se crean o modifican los objetos con los datos de ID, nombre y edad. Además, el controlador interactúa con Repositorio_estudiante, el cual almacena y devuelve la información actualizada de los estudiantes.

Finalmente, la vista refresca la tabla en tiempo real para mostrar los cambios realizados por el usuario dentro del sistema.

DIAGRAMA DE CLASES

Código PlantUML:

```
@startuml
title Diagrama de Clases de Diseño Purificado (U1T2)

class EstudianteService {
    - repositorio: Repositorio_estudiante
    --
    + EstudianteService(vista: Object, repositorio:
Repositorio_estudiante)
    + agregar(): void
    + actualizar(): void
    + visualizar(): void
    + eliminar(): void
}

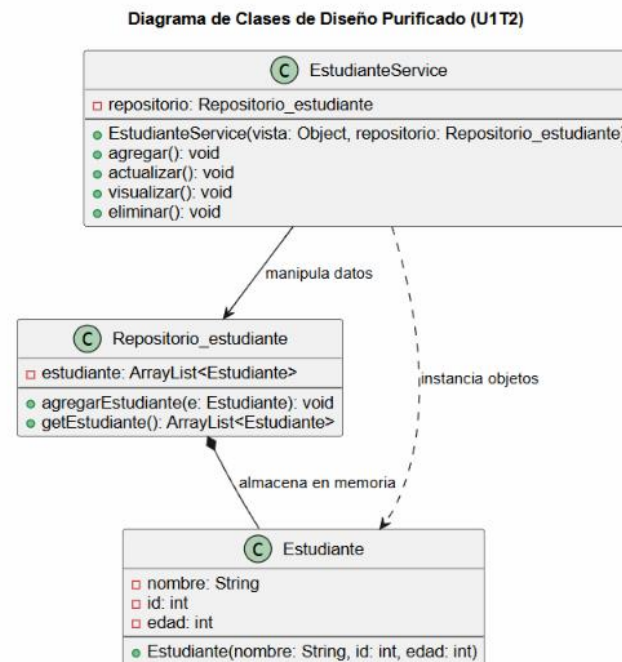
class Repositorio_estudiante {
    - estudiante: ArrayList<Estudiante>
    --
    + agregarEstudiante(e: Estudiante): void
    + getEstudiante(): ArrayList<Estudiante>
}

class Estudiante {
    - nombre: String
    - id: int
    - edad: int
    --
    + Estudiante(nombre: String, id: int, edad: int)
}

' Relaciones estructurales directas de la lógica de negocio
EstudianteService --> Repositorio_estudiante : manipula datos
EstudianteService ..> Estudiante : instancia objetos
Repositorio_estudiante *-- Estudiante : almacena en memoria

@enduml
```

IMAGEN DEL DIAGRAMA



Interpretación:

El diagrama de clases representa la estructura principal del sistema CRUD de estudiantes y la relación entre sus clases principales: **EstudianteService**, **Repositorio_estudiante** y **Estudiante**.

La clase **EstudianteService** actúa como controlador del sistema, ya que contiene los métodos principales para agregar, actualizar, visualizar y eliminar estudiantes. Además, se encarga de manipular los datos y comunicarse con el repositorio.

La clase **Repositorio_estudiante** funciona como almacenamiento en memoria utilizando una lista de tipo `ArrayList<Estudiante>`, donde se guardan los objetos estudiante. Esta clase posee métodos para agregar estudiantes y obtener la lista completa de registros.

Por otro lado, la clase **Estudiante** representa la entidad principal del sistema y contiene los atributos básicos: nombre, id y edad. También incluye un constructor para crear nuevos objetos estudiante.

Las relaciones del diagrama muestran que **EstudianteService** manipula los datos del repositorio y crea instancias de la clase **Estudiante**, mientras que el repositorio almacena los objetos creados en memoria para su posterior uso dentro del sistema.

10. Matriz de trazabilidad RF-CU para el ejemplo

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Agregar estudiante	Administrador académico	Alta	El sistema guarda un estudiante con ID único, Nombre obligatorio y Edad válida.	UC1
RF-02	Actualizar estudiante	Administrador académico	Alta	El sistema modifica Nombre y/o Edad luego de validar campos obligatorios.	UC2
RF-03	Eliminar estudiante	Administrador académico	Media	El sistema elimina o marca como eliminado al estudiante seleccionado.	UC3
RF-04	Mostrar todo	Administrador académico	Alta	El sistema muestra una tabla con las columnas ID, Nombre y Edad.	UC4
RF-05	Validar datos del estudiante	Sistema	Alta	El sistema impide guardar o actualizar registros con ID vacío, nombre vacío o edad inválida.	UC5

Conclusiones

1. El desarrollo del sistema CRUD de estudiantes permitió comprender la importancia de la gestión y organización de datos dentro de una aplicación académica utilizando programación orientada a objetos.
2. Mediante los diagramas UML, como casos de uso, robustez y clases, se logró representar de forma clara la estructura, comportamiento e interacción de los componentes del sistema.
3. El uso de las operaciones CRUD permitió implementar funciones básicas como agregar, actualizar, eliminar y visualizar estudiantes, facilitando el manejo de la información académica.
4. La validación de datos demostró ser un proceso importante para garantizar que la información ingresada por el usuario sea correcta y evitar errores dentro del sistema.
5. Este trabajo ayudó a fortalecer conocimientos en modelado de software, análisis de requisitos y diseño de sistemas utilizando herramientas como PlantUML y lenguaje Java.

Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.
Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.